

Universidad de Costa Rica

Sede del Pacífico

Campus Arnoldo Ferreto Segura Pacífico

Curso: Algoritmos y Estructura de Datos

Proyecto:

Algoritmos recursivos

Juego de la Vida - Ecosistema costarricense

Integrantes:

David Mora Gómez - C5H441

Daniela Bustos Medina - C5D410

Jimena Conejo Artavia - C5E399

Profesor:

Karol Sugeil Sánchez Pérez

Junio, 2026

Índice

Portada	1
Índice	2
1. Objetivos	3
2. Introducción	4
3. Desarrollo	5
3.1 Concepto de recursividad	5
3.2 Caso base, caso recursivo y pila de llamadas	6
3.3 Recursividad e iteración	6
3.4 Ejemplos cotidianos	7
3.5 Ejemplos empresariales y de sistemas	8
3.6 Recursividad en estructuras de datos	9
3.7 Ventajas, riesgos y buenas prácticas	10
3.8 Aplicación en el proyecto programado	11
3.9 Explicación y validación del método recursivo	12
4. Conclusiones	15
5. Referencias bibliográficas	16

1. Objetivos

1.1 Objetivo general

Analizar los algoritmos recursivos y su uso dentro del Juego de la Vida de Conway, para tener una base clara antes de programar la simulación de microorganismos en un ecosistema costarricense.

1.2 Objetivos específicos

- Explicar qué es la recursividad y por qué necesita un caso base.
- Analizar cómo la recursividad permite dividir problemas complejos en pasos más pequeños.
- Identificar ejemplos de recursividad en situaciones cotidianas, empresariales y de sistemas.
- Justificar el método recursivo usado para contar vecinos vivos en la aplicación del proyecto.

Estos objetivos se enfocan primero en la recursividad como tema principal. El Juego de la Vida se mantiene como aplicación final del proyecto, pero no como el centro completo de la investigación.

2. Introducción

Los algoritmos recursivos son una forma de resolver problemas haciendo que un método se llame de nuevo a sí mismo, pero con una parte más pequeña del problema original. Esta idea puede parecer extraña al inicio, porque uno normalmente piensa en resolver las cosas con pasos seguidos o con ciclos. Sin embargo, la recursividad aparece cuando una tarea se puede repetir dentro de sí misma hasta llegar a una condición sencilla de terminar.

La recursividad no se limita a ejemplos matemáticos. También se puede observar en carpetas dentro de carpetas, menús de sistemas, árboles familiares, organigramas de empresas, productos compuestos por piezas y reportes que se agrupan por niveles. Como por ejemplo, una empresa puede calcular el costo total de un producto revisando sus componentes, luego los componentes de esos componentes, y así hasta llegar a piezas simples.

En esta investigación se estudia la recursividad como tema central del grupo 4. El Juego de la Vida de Conway se incluye porque es la aplicación programada solicitada en el proyecto, pero la teoría se desarrolla principalmente alrededor de los algoritmos recursivos, sus partes, sus ventajas, sus riesgos y sus usos reales. Para respaldar el trabajo se usan libros, artículos académicos, documentación oficial y fuentes institucionales; no se utiliza Wikipedia.

3. Desarrollo

3.1 Concepto de recursividad

La recursividad es una técnica de programación donde una función o método se llama a sí mismo para resolver una versión más pequeña del mismo problema. Black y Rodgers (2013) la presentan como una técnica algorítmica en la que una función cumple una tarea llamándose con una parte de esa tarea. En palabras más sencillas, no se intenta resolver todo de una vez, sino que se reduce el problema y se repite la misma lógica hasta llegar a una respuesta directa.

Este concepto se puede entender mejor si se piensa en un problema que se parece a sí mismo. Por ejemplo, una carpeta puede contener archivos y también otras carpetas. Cada subcarpeta se revisa de una manera muy parecida a la carpeta principal. Ese patrón repetido es lo que permite usar recursividad, porque el procedimiento general no cambia: revisar lo que hay dentro, entrar a las subcarpetas y detenerse cuando ya no queden más niveles.

Roberts (2006) explica que pensar recursivamente exige aceptar que el mismo procedimiento puede resolver una parte más pequeña del trabajo. Esta idea al principio no siempre se siente natural, porque uno quiere ver todos los pasos de forma lineal. Aun así, cuando el problema tiene niveles o se divide en partes parecidas, la recursividad ayuda a escribir una solución más cercana a la forma real del problema.

Un ejemplo clásico es el factorial. Para calcular $5!$, se puede expresar como 5 multiplicado por $4!$, y $4!$ como 4 multiplicado por $3!$, hasta llegar a 1. Lo importante de ese ejemplo no es solo la operación matemática, sino la reducción progresiva. Cada llamada trabaja con un número menor, de modo que el proceso se acerca a un punto donde ya se puede contestar sin llamar de nuevo al método.

La recursividad también aparece en problemas de búsqueda. Si se quiere encontrar un dato dentro de una estructura ordenada, como en la búsqueda binaria, el algoritmo puede revisar la mitad del espacio y luego repetir la búsqueda solo en la mitad que todavía puede contener el valor. Cormen et al. (2022) tratan este tipo de diseño como parte de las estrategias para dividir problemas en subproblemas más manejables.

No todos los problemas necesitan recursividad. Si una tarea es simplemente repetir cien veces una instrucción, un ciclo puede ser más claro. La recursividad se justifica mejor cuando el problema se define por niveles, ramas, partes internas o casos que se parecen al caso original. Por eso, antes de programar, conviene preguntarse si el problema realmente tiene esa estructura repetida.

En programación, una llamada recursiva no crea una magia extraña. El lenguaje solo ejecuta el mismo método otra vez con otros datos. La diferencia está en que cada llamada conserva su propio contexto: sus parámetros, variables locales y punto de retorno. Esa es la razón por la cual un método recursivo puede bajar hasta un caso simple y luego regresar resultados poco a poco.

Para este proyecto, el tema principal no es solamente el Juego de la Vida, sino la recursividad como forma de construir algoritmos. El juego sirve como aplicación práctica porque obliga a contar vecinos y aplicar reglas sobre una matriz. Sin embargo, la investigación debe dejar claro que el objetivo académico es comprender cómo se plantea, se detiene y se defiende un método recursivo.

3.2 Caso base, caso recursivo y pila de llamadas

Todo algoritmo recursivo necesita un caso base. El caso base es la condición que detiene las llamadas recursivas. Si no existe, el método se llama a sí mismo sin final y el programa termina fallando por exceso de llamadas. En una explicación sencilla, el caso base es el momento en que el problema ya está tan reducido que se puede responder directamente.

El caso recursivo es la parte donde el método vuelve a llamarse, pero con datos más cercanos al caso base. En el factorial, el caso recursivo de factorial(n) llama a factorial(n - 1). En una búsqueda dentro de carpetas, el caso recursivo aparece cuando el método encuentra una subcarpeta y decide revisarla con el mismo procedimiento. En ambos casos, la llamada cambia el dato de entrada.

Si el caso recursivo no reduce el problema, la solución queda mal planteada. Por ejemplo, si factorial(5) llamara otra vez a factorial(5), nunca llegaría a factorial(1). Ese error es común cuando se empieza a programar recursividad, porque la persona ve que el método se llama a sí mismo, pero no revisa si la llamada realmente avanza hacia la salida.

La pila de llamadas permite entender qué ocurre por dentro. Cada vez que un método llama a otro método, el programa guarda información de la llamada anterior para poder continuar después. En recursividad pasa lo mismo, solo que el método llamado es el mismo. Cada llamada queda esperando el resultado de la siguiente. Cuando se alcanza el caso base, las respuestas regresan en orden inverso.

Este regreso en orden inverso es una parte que se debe explicar bien en la exposición. Si se calcula una suma recursiva, primero se baja hasta el último caso y luego se van devolviendo los resultados. No es que todas las llamadas se resuelvan al mismo tiempo. Cada llamada queda pendiente y solo termina cuando la llamada que hizo también termina.

En Java, una recursión demasiado profunda puede producir un error de desbordamiento de pila. Esto ocurre porque la memoria reservada para las llamadas no es infinita. Por esa razón, la recursividad debe usarse con cuidado. No basta con que el método funcione para un ejemplo pequeño; también hay que pensar si el tamaño del problema puede crecer mucho.

Una forma práctica de revisar un método recursivo es escribir una traza. La traza anota los valores de los parámetros en cada llamada. Por ejemplo, si el método recibe un índice, se puede escribir índice 0, índice 1, índice 2 y así hasta el caso base. Esa tabla o explicación ayuda a demostrar que el método avanza y que no está repitiendo el mismo estado.

La relación entre caso base y caso recursivo también sirve para detectar errores de lógica. Si el caso base está antes de tiempo, el método puede detenerse sin revisar todo. Si está muy tarde, puede intentar entrar a una posición inválida o hacer una llamada innecesaria. Por eso, al programar, se debe pensar primero cuándo termina y después cómo se mueve la llamada.

En el conteo de vecinos del proyecto, el caso base será llegar al índice 8, porque hay ocho posiciones vecinas. El caso recursivo revisará una posición y llamará al método con índice + 1. Esta estructura es defendible porque el índice siempre aumenta y porque, al llegar a 8, ya no queda ninguna dirección por evaluar.

3.3 Recursividad e iteración

La recursividad y la iteración sirven para repetir procesos, pero no se expresan igual. La iteración utiliza ciclos como for o while. La recursividad utiliza llamadas al mismo método. En

muchos casos, un problema puede resolverse de las dos formas. La pregunta importante no es cuál suena más avanzada, sino cuál describe mejor el problema y cuál resulta más clara para explicarlo.

La iteración suele ser más directa cuando la repetición es lineal. Por ejemplo, recorrer una lista de diez estudiantes e imprimir sus nombres se entiende muy bien con un ciclo. No hay niveles internos ni ramas. En cambio, si cada estudiante tuviera una lista de cursos, y cada curso una lista de temas, la estructura empieza a tener niveles y la recursividad puede resultar más natural.

Sedgewick y Wayne (2011) explican muchos algoritmos donde la elección de estructura influye en la claridad de la solución. En árboles, por ejemplo, cada nodo puede tener hijos y cada hijo puede tener otros hijos. Para recorrerlos, se puede visitar un nodo y luego aplicar el mismo recorrido a sus hijos. Esa forma de pensar se adapta bastante bien a la recursividad.

La iteración también puede resolver estructuras jerárquicas, pero a veces necesita pilas o colas auxiliares. Eso significa que el programador debe administrar manualmente qué parte falta por revisar. La recursividad, en cambio, deja que la pila de llamadas maneje ese orden. Esto puede hacer el código más corto, aunque no siempre más eficiente.

Una desventaja de la recursividad es el consumo de memoria. Cada llamada guarda información. Si hay muchas llamadas pendientes, el programa puede fallar. La iteración suele gastar menos memoria en problemas simples, porque repite instrucciones dentro del mismo contexto. Por eso, para tareas muy grandes o muy lineales, un ciclo puede ser una decisión más segura.

Otra diferencia es la facilidad para depurar. En un ciclo, se revisa el valor de una variable que cambia en cada vuelta. En recursividad, se revisan varias llamadas con sus propios valores. Esto puede confundir al inicio. Por eso, en trabajos académicos conviene acompañar la explicación con un ejemplo pequeño, como contar ocho vecinos o sumar una lista corta.

En el proyecto, la profesora solicita específicamente que el conteo de vecinos se realice con recursividad. Eso significa que, aunque un ciclo sería una solución común, el objetivo es demostrar el tema asignado. El programa puede usar ciclos para recorrer toda la matriz, pero el núcleo solicitado, revisar las ocho posiciones vecinas, debe hacerse con un método recursivo.

Esta distinción es importante: no todo el programa tiene que ser recursivo. La interfaz, la lectura de datos y la actualización de la matriz pueden usar estructuras normales. La recursividad se aplica donde tiene sentido según el requisito. Así el código no se vuelve forzado y la explicación mantiene coherencia con el tema del grupo.

3.4 Ejemplos cotidianos

Un ejemplo cotidiano de recursividad es buscar un documento dentro de una carpeta que puede tener subcarpetas. Primero se revisa la carpeta principal. Si aparece una subcarpeta, se revisa esa subcarpeta con el mismo procedimiento. Luego, si dentro de esa subcarpeta hay otra, se repite el proceso. La búsqueda termina cuando ya no quedan carpetas por abrir o cuando se encuentra el archivo.

Otro ejemplo se puede ver en los árboles familiares. Para conocer los antepasados de una persona, se revisan sus padres; después se revisan los padres de sus padres, y así sucesivamente.

La pregunta se repite en cada nivel: quiénes son los padres de esta persona. El proceso termina cuando ya no se tiene información de generaciones anteriores.

También se puede pensar en una receta dividida en pasos. Algunas recetas tienen preparaciones internas, como una salsa que también tiene su propio proceso. Para preparar el plato completo, se ejecuta la receta principal, pero cuando se llega a la salsa, se sigue otra receta más pequeña. Esa receta interna puede tener otra preparación y así seguir hasta llegar a pasos simples.

En una conversación digital también puede aparecer esta idea. Un comentario puede tener respuestas, y esas respuestas pueden tener más respuestas. Si una aplicación quiere mostrar todo el hilo, debe mostrar el comentario principal y luego aplicar el mismo proceso a cada respuesta. La estructura no es una lista plana; es una cadena de niveles.

Como por ejemplo, cuando una persona organiza una mudanza, puede dividir la casa en habitaciones, luego cada habitación en muebles, y cada mueble en cajas o partes pequeñas. El método mental es recursivo porque se toma una unidad grande y se repite el mismo criterio de división hasta llegar a algo manejable. Aunque no se esté programando, la lógica de reducción es parecida.

Estos ejemplos ayudan porque muestran que la recursividad no es solo una palabra técnica. Es una forma de resolver situaciones donde algo contiene otra cosa parecida. En programación, esa idea se formaliza con métodos, parámetros, casos base y retornos. Pero la lógica general ya existe en muchas actividades de la vida diaria.

3.5 Ejemplos empresariales y de sistemas

En una empresa, la recursividad puede aparecer en un organigrama. Una gerencia puede tener varios departamentos, cada departamento puede tener equipos, y cada equipo puede tener colaboradores. Si se quiere contar cuántas personas dependen de una gerencia, se puede contar cada departamento y repetir el mismo proceso con los niveles internos. Este tipo de recorrido se adapta bien a una solución recursiva.

Otro ejemplo empresarial está en las listas de materiales, conocidas como estructuras de producto. Un producto terminado puede estar formado por piezas, y algunas piezas pueden estar formadas por subpiezas. Para calcular el costo total del producto, el sistema revisa cada componente y, si ese componente también tiene partes internas, vuelve a aplicar el mismo cálculo. Esto se parece mucho a recorrer un árbol.

Los sistemas de reportes también pueden usar ideas recursivas. Una compañía puede tener reportes por país, provincia, sucursal, departamento y empleado. Si se desea sumar ventas totales, el sistema puede sumar primero los niveles más pequeños y luego subir esos resultados a niveles superiores. Aunque el usuario solo ve un total, internamente el proceso puede repetirse por cada nivel de la estructura.

En páginas web y sistemas administrativos, los menús anidados son otro caso común. Un menú principal puede tener opciones, y cada opción puede abrir subopciones. Para dibujar el menú completo, el sistema puede recorrer cada opción y, si tiene hijos, dibujarlos usando el mismo procedimiento. Este tipo de problema se entiende mejor cuando se reconoce su forma recursiva.

La recursividad también se relaciona con permisos de usuario. En algunas organizaciones, un rol puede heredar permisos de otro rol, y ese rol puede heredar permisos adicionales. Para saber qué puede hacer una persona dentro de un sistema, el programa puede revisar el rol actual y luego revisar los roles de los que hereda. El proceso termina cuando no hay más roles superiores.

En comercio electrónico se puede observar en categorías de productos. Una tienda puede tener la categoría Tecnología, dentro de ella Computadoras, dentro de Computadoras Portátiles, y dentro de Portátiles Accesorios. Si el sistema quiere listar todos los productos de Tecnología, debe bajar por cada subcategoría y repetir el mismo procedimiento hasta llegar a las categorías finales.

En bases de datos, algunas consultas trabajan con relaciones jerárquicas. Por ejemplo, una tabla de empleados puede guardar el jefe de cada empleado. Para construir la cadena completa de mando, se empieza con una persona y se busca su jefe, luego el jefe del jefe, hasta llegar a la dirección general. Aunque la implementación depende del motor de base de datos, la idea de fondo es recursiva.

Estos ejemplos empresariales son útiles para el trabajo porque muestran que la recursividad no es un tema aislado de la clase. En informática empresarial, muchas soluciones se relacionan con estructuras organizadas por niveles. Entender recursividad ayuda a diseñar sistemas que procesan jerarquías de forma clara, desde un organigrama hasta una estructura de productos o permisos.

3.6 Recursividad en estructuras de datos

La recursividad tiene una relación fuerte con las estructuras de datos. Una lista simple se puede recorrer con ciclos, pero estructuras como árboles, carpetas, expresiones matemáticas y comentarios con respuestas suelen tener forma recursiva. Un árbol, por ejemplo, está formado por un nodo raíz y subárboles. Cada subárbol puede tratarse como un árbol más pequeño.

En informática, los árboles se usan en bases de datos, compiladores, sistemas de archivos, inteligencia artificial y estructuras de búsqueda. Para recorrer un árbol, muchas veces se visita un nodo y luego se aplica el mismo recorrido a sus hijos. Esta lógica puede escribirse con recursividad de forma clara, porque el código refleja la forma real de la estructura.

Knuth (1997) presenta los algoritmos como procedimientos que deben ser precisos, finitos y efectivos. Esa idea se relaciona con la recursividad porque un algoritmo recursivo también debe terminar. No basta con que la definición se vea elegante; debe existir una forma concreta de avanzar y detenerse. Esa condición de finitud es clave para defender cualquier método recursivo.

También se utiliza recursividad para procesar expresiones matemáticas. Una expresión como $(2 + (3 * 4))$ tiene partes dentro de partes. Un programa puede evaluar primero las subexpresiones y luego combinar los resultados. Este tipo de análisis aparece en compiladores e intérpretes, donde el sistema debe entender instrucciones escritas por una persona.

Otra estructura relacionada es el sistema de archivos. Una carpeta puede contener archivos y carpetas. A su vez, esas carpetas pueden contener más archivos y carpetas. Para calcular el tamaño total de una carpeta, se suma el tamaño de sus archivos y luego se suma el tamaño de cada subcarpeta aplicando el mismo proceso. Este ejemplo es bastante fácil de explicar en una exposición porque se parece a algo que todos usan.

La recursividad también aparece en algoritmos de ordenamiento. En estrategias como dividir y conquistar, se divide una colección en partes más pequeñas, se resuelve cada parte y luego se combina el resultado. Cormen et al. (2022) explican que este enfoque permite diseñar algoritmos importantes porque transforma un problema grande en subproblemas de la misma naturaleza.

En grafos, el uso de recursividad puede verse en búsquedas por profundidad. El algoritmo visita un nodo, marca que ya fue visitado y luego explora sus vecinos. Si un vecino no ha sido visitado, se llama de nuevo el procedimiento desde ese vecino. El caso base aparece cuando no quedan vecinos nuevos que explorar o cuando ya se alcanzó el objetivo buscado.

Los árboles de decisión también se pueden explicar de forma recursiva. Un nodo hace una pregunta, sus ramas llevan a nuevas preguntas y el proceso continúa hasta llegar a una hoja con una respuesta. En sistemas empresariales, esto puede aparecer en flujos de aprobación, evaluación de riesgos o clasificación de solicitudes. La estructura tiene niveles y cada nivel repite una lógica parecida.

La importancia de estos ejemplos es que conectan el tema con estructuras de datos reales. No se estudia recursividad solo para calcular factoriales. Se estudia porque muchas estructuras que maneja un sistema tienen una forma natural de niveles. Cuando se reconoce esa forma, el diseño del algoritmo se vuelve más ordenado y más fácil de explicar.

3.7 Ventajas, riesgos y buenas prácticas

Una ventaja de la recursividad es que puede hacer que ciertos algoritmos sean más claros. Cuando el problema tiene una estructura repetida, el código recursivo puede parecerse mucho a la explicación del problema. Esto ayuda a leerlo y defenderlo. Por ejemplo, decir 'revisar una carpeta y luego revisar sus subcarpetas' se traduce naturalmente en un método que se llama a sí mismo.

Otra ventaja es que permite dividir problemas grandes en partes más pequeñas. Esa división ayuda a concentrarse en una parte del problema y confiar en que el mismo método resolverá el resto. Esta idea es importante en algoritmos clásicos como búsqueda binaria, recorridos de árboles, ordenamientos recursivos y análisis de estructuras jerárquicas.

Sin embargo, la recursividad también tiene riesgos. Si no se define bien el caso base, el método no se detiene. Si cada llamada no reduce el problema, tampoco se avanza. Además, cada llamada consume espacio en la pila de ejecución. En problemas muy grandes, una recursión profunda puede causar errores de memoria o hacer que el programa sea menos eficiente.

Una buena práctica es escribir primero el caso base y después el caso recursivo. También conviene probar con ejemplos pequeños antes de usar datos grandes. Si el método funciona para un caso simple, luego se puede revisar con casos más completos. Además, los nombres de variables deben ser claros, porque en recursividad es fácil confundirse con los valores de cada llamada.

Otra buena práctica es explicar la recursividad con trazas. Una traza muestra qué llamada se ejecuta primero, cuál se ejecuta después y qué retorna cada una. Esto ayuda mucho para la defensa oral, porque permite demostrar que el grupo entiende el proceso interno. No basta con decir que el método se llama a sí mismo; hay que explicar cuándo se detiene y cómo se acumula el resultado.

También se debe evitar mezclar demasiadas responsabilidades en un método recursivo. Si un método cuenta, imprime, modifica la interfaz y cambia datos al mismo tiempo, será difícil revisarlo. Es mejor que el método recursivo tenga una tarea concreta. En el caso del proyecto, esa tarea será contar vecinos vivos. La actualización visual puede hacerse en otra parte del programa.

Otra recomendación es validar los límites antes de acceder a una posición. Cuando se trabaja con matrices, una fila negativa o una columna fuera del tamaño puede generar errores. Por eso, el método debe revisar si la posición vecina está dentro de la matriz. Si está fuera, no suma nada y continúa con la siguiente dirección recursiva.

Finalmente, la recursividad debe explicarse con lenguaje sencillo. En una defensa oral, puede ser más claro decir que el método revisa una dirección por llamada, avanza el índice y se detiene al llegar a ocho. Esa explicación comunica la lógica sin complicarla. Luego, si la profesora pregunta detalles, se puede mencionar el caso base, el caso recursivo y la pila de llamadas.

3.8 Aplicación en el proyecto programado

El Juego de la Vida queda como la aplicación práctica del proyecto. Gardner (1970) presentó este modelo como un conjunto de celdas vivas y vacías que cambian de estado según reglas locales. En el programa asignado al grupo, cada celda representa una zona de un ecosistema costarricense y cada microorganismo puede vivir, morir o nacer según sus vecinos.

Aunque el juego usa una matriz y reglas de supervivencia, el tema del grupo sigue siendo la recursividad. Por eso, la parte más importante no es solamente dibujar la matriz, sino explicar cómo se cuenta la cantidad de vecinos vivos. El requisito técnico pide que ese conteo se haga con un método recursivo y no con ciclos para recorrer las ocho posiciones vecinas.

Wolfram (1983) estudia los autómatas celulares como sistemas donde reglas locales pueden generar comportamientos globales. Esa idea ayuda a entender por qué el Juego de la Vida es útil en un curso de algoritmos. Cada celda solo revisa su vecindad, pero la matriz completa cambia generación tras generación. El resultado depende de muchas decisiones pequeñas aplicadas de forma consistente.

La solución propuesta consiste en guardar las ocho direcciones vecinas en un arreglo. El método recibe la fila, la columna y un índice. En cada llamada revisa una dirección. Si la posición está dentro de la matriz y tiene un microorganismo vivo, suma uno. Después llama al mismo método con el índice aumentado. Cuando el índice llega a ocho, se alcanza el caso base y el método retorna cero.

La dirección (0,0) no se incluye porque representa la misma celda que se está evaluando. Si se contara esa posición, una celda viva podría contarse como vecina de sí misma, lo cual sería incorrecto. Esta explicación es una de las más importantes para la defensa oral, porque muestra que el grupo entiende tanto la regla del juego como la estructura recursiva del método.

El programa también debe separar la generación actual de la generación siguiente. Si se actualiza la matriz mientras todavía se están evaluando celdas, una celda podría afectar indebidamente a otra dentro de la misma generación. Por eso, se recomienda calcular los cambios en una matriz auxiliar y luego reemplazar la matriz principal cuando ya se evaluaron todas las celdas.

La ambientación costarricense puede manejarse con nombres de áreas silvestres protegidas. El Sistema Nacional de Áreas de Conservación (s. f.) presenta información institucional sobre estas áreas, lo cual permite usar un contexto nacional sin inventar lugares. La simulación no pretende ser un modelo biológico exacto, sino una representación educativa de microorganismos dentro de una reserva.

Oracle (s. f.) documenta Swing como una biblioteca para construir interfaces gráficas en Java. Usar Swing o JavaFX es permitido porque la interfaz no resuelve el núcleo lógico del algoritmo. La lógica central sigue estando en el conteo recursivo de vecinos y en la aplicación de reglas. La interfaz solo facilita que el usuario vea la matriz, seleccione tamaños y observe las generaciones.

Para explicar el método en clase, el grupo puede usar una matriz pequeña de 4x4. Se toma una celda, se muestran sus ocho posibles vecinos y se recorre el arreglo de direcciones con índice 0, 1, 2, hasta 7. Después se indica que al llegar a 8 el método se detiene. Esa demostración sencilla permite comprobar que la recursividad no se está usando de adorno, sino para cumplir una función concreta.

3.9 Explicación y validación del método recursivo

Para defender el algoritmo, conviene explicar primero qué problema resuelve el método. El método no genera toda la matriz, no decide todas las reglas y no dibuja la interfaz. Su responsabilidad concreta es contar cuántos vecinos vivos tiene una celda. Esa separación es importante porque evita mezclar la lógica recursiva con otras partes del programa. Cuando una función tiene una responsabilidad clara, también se vuelve más fácil probarla y corregirla.

La explicación puede iniciar con las ocho direcciones posibles alrededor de una celda. Se pueden escribir como pares de cambios: (-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0) y (1,1). Cada par indica cómo moverse desde la celda actual hacia una posición vecina. La pareja (0,0) queda fuera porque no mueve la fila ni la columna; por lo tanto, representa la misma celda.

Después se puede explicar el parámetro índice. El índice indica cuál dirección se revisa en cada llamada. Si el índice vale 0, se revisa la primera dirección. Si vale 1, se revisa la segunda. El proceso continúa hasta llegar a 8. En ese momento ya no hay más direcciones por revisar, de modo que el método retorna cero. Ese retorno es el caso base.

El caso recursivo suma el resultado de la dirección actual con el resultado de las demás direcciones. Si la posición vecina está dentro de la matriz y contiene un microorganismo vivo, se suma 1. Si está fuera de la matriz o está vacía, se suma 0. Luego el método llama de nuevo con índice + 1. Así, el total final se construye con pequeñas sumas parciales.

Un caso borde ocurre cuando la celda está en una esquina. Por ejemplo, la celda (0,0) no tiene vecinos arriba ni a la izquierda, porque esas posiciones quedarían fuera de la matriz. El método recursivo debe revisar los límites antes de leer la celda vecina. Si la fila calculada es menor que cero o mayor que el último índice válido, no se suma nada y se continúa con la siguiente llamada.

Otro caso borde aparece en los lados de la matriz. Una celda de la primera fila puede tener vecinos a la izquierda, derecha y abajo, pero no arriba. Esto no cambia la estructura recursiva. El método sigue recorriendo las ocho direcciones; simplemente algunas no se cuentan porque están fuera de los límites. Esa decisión mantiene el algoritmo uniforme para esquinas, bordes y celdas internas.

Para probar el método, se puede usar una matriz 4x4 con pocos microorganismos. Primero se calcula manualmente cuántos vecinos vivos tiene una celda específica. Luego se ejecuta el método y se compara el resultado. Si ambos resultados coinciden, se prueba otra celda en una esquina y otra en el centro. Esta forma de validación es sencilla y permite detectar errores antes de ejecutar varias generaciones.

También se debe probar que el método no cuenta la celda actual. Si una celda está viva y no tiene vecinos, el resultado debe ser cero, no uno. Esta prueba es importante porque uno de los errores más fáciles sería incluir la dirección (0,0) dentro del arreglo. Si eso sucede, la regla de supervivencia se altera y el comportamiento del juego cambia de forma incorrecta.

La complejidad del conteo de vecinos es constante, porque siempre se revisan ocho direcciones. Aunque se use recursividad, la cantidad de llamadas para una celda no crece con el tamaño de la matriz. Lo que sí crece es la cantidad de celdas por evaluar en cada generación. Por eso, el programa completo depende del tamaño de la matriz y de la cantidad de generaciones indicadas por el usuario.

En la defensa oral se puede mencionar que la recursividad se usa por requisito técnico y por claridad didáctica. Un ciclo sería posible, pero el objetivo del tema 4 es demostrar que el grupo sabe construir un método recursivo con caso base, caso recursivo y avance de parámetros. Esa explicación muestra criterio, porque reconoce la alternativa iterativa sin dejar de cumplir lo solicitado.

Por último, la validación debe incluir el resultado de cada generación. El programa tiene que indicar cuántos microorganismos sobreviven, cuántos nacen y cuántos mueren. Estos datos no salen solamente del conteo recursivo, sino de aplicar las reglas después de contar. Aun así, si el conteo de vecinos está mal, todo el resumen de la generación queda mal. Por eso, esa parte del algoritmo merece una explicación detallada.

Un error común sería actualizar la celda apenas se calcula su nuevo estado. Eso no debe hacerse, porque las demás celdas todavía necesitan leer la generación anterior. La solución más segura es usar una matriz nueva para guardar el siguiente estado. Cuando todas las celdas fueron revisadas, la matriz nueva reemplaza a la anterior. Esta separación mantiene la lógica correcta de generación por generación.

También es útil diferenciar entre contar vecinos y aplicar reglas. El método recursivo solo entrega un número. Luego otro método decide si la celda vive, muere o nace según ese número y el estado actual. Esta división hace que el código sea más ordenado, porque cada método tiene una tarea concreta. Además, si hay un error, se puede revisar una parte sin culpar a todo el programa.

La documentación del código debe explicar el caso base con palabras claras. Por ejemplo, se puede comentar que si el índice es igual a la cantidad de direcciones, ya no quedan vecinos por revisar. Ese comentario no reemplaza la explicación oral, pero ayuda a que el lector entienda la intención. En un proyecto académico, esa claridad cuenta porque la profesora puede pedir una defensa línea por línea.

El método también debe retornar un valor, no solo imprimirlo. Si solo imprime el conteo, otras partes del programa no podrían usarlo para aplicar las reglas. Al retornar el número de vecinos vivos, el método se vuelve reutilizable. La misma función puede usarse para una celda del centro, una esquina, un borde o cualquier tamaño de matriz permitido por el enunciado.

Para revisar el comportamiento, se puede preparar una tabla pequeña con tres columnas: celda evaluada, vecinos vivos esperados y resultado del método. Esta tabla no necesita ser parte de la interfaz, pero puede servir para pruebas internas del grupo. Si el resultado esperado y el resultado del método coinciden en varias posiciones, hay más confianza en que la recursividad está bien implementada.

Otra forma de explicar el algoritmo es describirlo como una pregunta repetida. La pregunta sería: ¿esta dirección tiene un vecino vivo? Luego se hace la misma pregunta para la siguiente dirección. Cuando ya se preguntó por las ocho direcciones, se termina. Esta manera de explicarlo suena menos pesada que decir solamente 'función que se llama a sí misma', y ayuda a que el público entienda el propósito del método.

En síntesis, la validación del método recursivo debe revisar tres puntos: que el índice avance, que el caso base se alcance y que el conteo respete los límites de la matriz. Si esos tres puntos se cumplen, el método tiene una estructura defendible. Luego, las reglas del Juego de la Vida pueden aplicarse con mayor seguridad, porque el dato principal para decidir cada cambio ya fue calculado correctamente.

Una prueba adicional consiste en usar una celda vacía con exactamente tres vecinos vivos. En ese caso, el resultado del conteo recursivo debe ser tres y la regla posterior debe producir un nacimiento. Si el conteo devuelve dos o cuatro, el error no está en la regla de nacimiento, sino en la forma en que se revisaron las posiciones vecinas. Separar esos dos pasos ayuda a encontrar fallos con más rapidez.

También se puede probar una celda viva con más de tres vecinos. Según las reglas, esa celda debe morir por asfixia. Para revisar el algoritmo, primero se confirma que el método recursivo cuente cuatro o más vecinos vivos. Después se revisa que la regla cambie el estado de la celda a vacía. Esta prueba conecta directamente la teoría del conteo con el resultado mostrado en la generación siguiente.

En el reporte del programa, conviene mostrar los tres grupos de resultados: nacimientos, sobrevivientes y muertes. Esa salida ayuda al usuario a entender qué pasó en cada generación y no solo a ver una matriz final. Además, permite defender que el programa no está cambiando celdas al azar, sino aplicando reglas claras a partir de un conteo previo.

El método recursivo debe manejar cualquier tamaño permitido por el enunciado, desde 4x4 hasta 8x8. Esto significa que no se debe programar pensando únicamente en una matriz fija. La validación de límites debe usar el tamaño real de la matriz. Así, el mismo método funciona para todos los casos sin copiar código ni crear soluciones diferentes para cada tamaño.

Otro punto importante es que la recursividad debe estar en el núcleo lógico, no en la interfaz. La interfaz puede tener botones, cuadros de texto o una matriz visual, pero esas partes no demuestran por sí solas el tema asignado. Lo que demuestra el tema es el método que se llama a sí mismo para revisar las direcciones vecinas y retornar el total de microorganismos alrededor de una celda.

De esta manera, el proyecto une teoría y práctica. La investigación explica qué es la recursividad, por qué necesita un caso base y cómo aparece en situaciones reales. El programa toma esa explicación y la aplica en un punto concreto: contar vecinos vivos. Esa relación es importante porque evita que el documento escrito y el código parezcan dos trabajos separados.

4. Conclusiones

La recursividad es una herramienta útil cuando un problema puede dividirse en partes más pequeñas que se parecen al problema original. Su valor principal está en que permite repetir una misma idea hasta llegar a un caso base claro. Por eso, comprender el caso base y el caso recursivo es más importante que memorizar una línea de código. Si el método no sabe cuándo detenerse o no reduce el problema, la solución deja de ser correcta aunque parezca bien escrita.

Los ejemplos cotidianos y empresariales ayudan a entender que la recursividad no es solo un tema de programación. Aparece en carpetas, organigramas, árboles familiares, productos compuestos, permisos, menús y reportes con niveles. En todos esos casos existe una estructura que se repite dentro de otra estructura, y eso permite aplicar una solución ordenada. Esta relación con situaciones reales facilita explicar el tema en clase sin quedarse únicamente en ejemplos matemáticos.

El Juego de la Vida funciona como aplicación práctica del tema asignado. En este caso, la recursividad se usa para contar vecinos vivos sin recorrer las ocho posiciones con ciclos. Así se cumple el requisito técnico y se demuestra cómo una idea teórica puede convertirse en una solución programada dentro de una matriz. El juego no debe verse como el centro de toda la investigación, sino como el escenario donde se aplica el algoritmo recursivo.

Para la defensa oral, lo más importante será explicar por qué el método se detiene, qué cambia en cada llamada y por qué la celda central no se cuenta como vecina. Si esos puntos se dominan, el grupo puede demostrar que entiende la recursividad y no solo la parte visual del programa. También será útil mostrar una traza pequeña con índices del 0 al 8, porque eso deja claro el avance del método y el retorno del resultado.

5. Referencias bibliográficas

- Adamatzky, A. (Ed.). (2010). Game of Life cellular automata. Springer London.
- Black, P. E., & Rodgers, P. (2013, 30 de septiembre). Recursion. En P. E. Black (Ed.), Dictionary of algorithms and data structures. National Institute of Standards and Technology.
<https://xlinux.nist.gov/dads/HTML/recursion.html>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- Gardner, M. (1970). Mathematical games: The fantastic combinations of John Conway's new solitaire game Life. Scientific American, 223(4), 120-123.
<https://www.scientificamerican.com/article/mathematical-games-1970-10/>
- Ilachinski, A. (2001). Cellular automata: A discrete universe. World Scientific.
- Knuth, D. E. (1997). The art of computer programming: Volume 1: Fundamental algorithms (3rd ed.). Addison-Wesley Professional. <https://www.informit.com/store/art-of-computer-programming-volume-1-fundamental-algorithms-9780133489330>
- Oracle. (s. f.). Trail: Creating a GUI With Swing. The Java Tutorials.
<https://docs.oracle.com/javase/tutorial/uiswing/index.html>
- Roberts, E. (2006). Thinking recursively with Java. John Wiley & Sons.
<https://cs.stanford.edu/people/eroberts/books/ThinkingRecursively/index.html>
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
<https://algs4.cs.princeton.edu/>
- Sistema Nacional de Áreas de Conservación. (s. f.). Áreas silvestres protegidas.
<https://www.sinac.go.cr/ES/asp/Paginas/default.aspx>
- Wolfram, S. (1983). Statistical mechanics of cellular automata. Reviews of Modern Physics, 55(3), 601-644. <https://content.wolfram.com/sw-publications/2020/08/statistical-mechanics-cellular-automata.pdf>